

Script Presentatore — Demo "Rome Future Week" (Crm-A Console)

Percorso consolidato della demo, beat per beat. Durata indicativa: **~30–32 min.**

Tono: vendita/educazione, ritmo **"touchpoint → record → campagna → chiamata vera → campagna inbound → riconoscimento"**.

Niente CURL sul palco. Ogni azione è una di queste tre cose:

1. un **comando in chat** all'agente (Copilot);
2. un **click in console / Shopify** (profilo, scheda campagna, checkout, dashboard NLPearl);
3. una **chiamata telefonica vera** (NLPearl fa l'AI della conversazione, la Console è il cervello CRM: riconosce, registra, orchestra).

I `curl` esistono solo come verifica tecnica **in preparazione** (sezione "Checklist pre-demo" + `scripts/shopify-demo-simulate.sh`), mai durante la demo.

Premessa narrativa: nessun contatto è pre-seeded. La demo parte **vuota**: il primo record nasce dal vivo, da un acquisto sull'e-commerce. Il touchpoint crea il cliente; da lì il CRM ricorda. I **servizi telefonici** (le Pearl NLPearl inbound e outbound) sono già **provisionati sul dashboard NLPearl**: la demo li **riusa per nome dalla chat** (Atto 1 e Atto 5) — l'agente aggancia la scheda e attiva la linea con una conferma, senza toccare la dashboard né creare nulla.

0. Apertura & setup (2 min) — "Da un touchpoint nasce un cliente"

 **Scena:** due tab affiancati: la Console (chat Copilot pronta) e lo **Shopify dev store** (`crm-a-demo-store`). Telefono del presentatore in vivavoce. Dashboard NLPearl in un altro tab.

 **Azione (solo prep, NON mostrata):** eseguire il **seed retail** + rimozione di Lorenzo con lo script dedicato (un solo comando, idempotente):

```
bash scripts/demo-seed.sh
# → seed (catalogo+persone+ordine+segmento) → rimuove Lorenzo → verifica
```

Dettaglio di ciò che crea il seed (`POST /api/demo/seed`) e di come si pulisce:

Tipo	Contenuto
Prodotti (3)	<code>SAM-S27</code> <i>Samsung Galaxy S27</i> — €1199, <i>Upcoming</i> , disponibile dal 2026-10-18, con messaggio marketing completo (chip +18%, batteria 5500 mAh, foto 200MP low-light, 7 anni update, promo €150 permuta) · <code>SAM-S26</code> — €999, <i>Available</i> · <code>SAM-S25</code> — €899, <i>Discontinued</i>
Persone (4)	<code>Lorenzo</code> (+393312345678, <code>lorenzo@example.com</code> , telegram, opt-in true) · <code>Giulia</code> (email, opt-in true) · <code>Marco</code> (telegram, opt-in true) · <code>Sara</code> (email, opt-in false)
Ordine (1)	ordine S26 di Lorenzo — <i>Shipped</i> , corriere <i>GLS</i>
Segmento (1)	<i>Lancio Samsung Galaxy</i> — filtro: <i>Marketing Opt-in = true</i>

Poi lo script **rimuove Lorenzo** (persona + il suo ordine seed, in quest'ordine) così il CRM parte senza di lui: il primo record nascerà dal vivo in Atto 0. Il catalogo, gli altri 3 contatti e il segmento restano.

Verifica alternativa (senza script):

```
curl -sX POST localhost:3100/api/demo/seed -H "Authorization: Bearer $CRM_A_PHONE_WEBHOOK_SECRET"
# → {"ok":true,"seeded":["products","people","order","segment"]}
# rimozione Lorenzo via chat: "Rimuovi il contatto Lorenzo" (persona + ordine)
```

Webhook Shopify configurato (`SHOPIFY_API_SECRET`); funnel attivo.

Mostra in console: Integrations → **card NLPearl** e **card Shopify** (URL webhook).

Poi:

(Mostra la card Shopify.) "Questo è il punto di ingresso: l'e-commerce. Quando qualcuno compra nel nostro store, un webhook arriva qui — e il CRM costruisce il profilo da zero. Nessun record esiste ancora. Vediamo cosa succede al primo acquisto."

👁️ **Pubblico vede:** Integrations con card NLPearl + shopify (webhook URL); nel DB nessun profilo "Lorenzo" pronto (vedi prep).

★ Atto 0 — Un acquisto crea il record (5 min) *(da qui il CRM ricorda)*

Obiettivo: dimostrare che **da un touchpoint nasce un cliente**: nessun contatto pre-caricato.

💻 Azione (live in Shopify):

1. In Shopify il presentatore completa il checkout **come cliente "Lorenzo"** (email `lorenzo@example.com`, telefono = il proprio numero) e paga il **Samsung Galaxy S26** (SKU `SAM-S26`).
2. Il webhook `orders/create` arriva alla Console → l'agente **crea il profilo** e registra l'ordine.
3. (*Beat memoria*) il presentatore segna l'ordine **evaso** in Shopify (corriere GLS, in transito) → webhook `order/fulfilled` → l'ordine riceve corriere + stato di consegna.
4. In console mostrare il **profilo appena creato** (nome, email, telefono) con la timeline: interazione `Purchase` + ordine S26.

Dettaglio del beat 3 ("segno l'ordine evaso"):

- **Dove si clicca:** nell'ordine di Lorenzo, **Shopify admin** → l'ordine → **"Fulfill item" / "Mark as fulfilled"**, aggiungendo la spedizione: **corriere GLS**, tracking number (es. `DEMO-S26`), stato `in_transit`.
- **Cosa succede dietro le quinte:** Shopify invia il webhook `order/fulfilled` a `/api/webhooks/shopify` (HMAC X-Shopify-Hmac-Sha256 con `SHOPIFY_API_SECRET`). La Console:
 1. **trova la persona** per email → telefono (**solo ricerca, mai creazione**: un fulfillment senza `orders/create` precedente non genera record fantasma);
 2. carica il suo **ultimo ordine** (`loadLastOrder`);
 3. **aggiorna i campi dell'ordine**: `Courier` = GLS, `Tracking URL`, e `Delivery Status` = "Corriere in carico oggi; consegna prevista domani entro le 18." (testo derivato da `in_transit` in `deliveryTextFromFulfillment`);
 4. aggiorna `Last Interaction At` sulla persona.

- **Cosa vede il pubblico:** in console l'ordine S26 ora ha **Courier:** GLS + **Delivery Status** compilati — è la **memoria** che la Pearl inbound pronuncerà nell'Atto 6.

Script:

"Guardate: il CRM era vuoto. Ora qualcuno acquista il Galaxy S26 — ed ecco cosa succede: il webhook confronta i campi — email, telefono, nome — non c'è nessuno, quindi **crea la nuova anagrafica**, registra l'evento **Purchase** e materializza l'ordine. Un secondo, e quello che era un checkout è un cliente con la sua storia. Poi segno la spedizione in Shopify: il webhook aggiorna corriere e consegna. **Questo è il punto: da un touchpoint nasce un record, e da qui il CRM ricorda.**"

(Abilitazione al consenso — il webhook porta SOLO l'identità, mai il consenso.)

"Nota: il traffico e-commerce ci ha dato solo i dati anagrafici. Il consenso al marketing lo decide l'operatore. In chat: 'Abilita Lorenzo alla campagna telefonica: consenso marketing sì, canale preferito telefono.' L'essere umano resta nel loop."

👁️ **Pubblico vede:** checkout in Shopify → in Console il profilo "Lorenzo" creato al volo (matched: created), timeline con **Purchase** + ordine S26 (GLS in transito); il comando di abilitazione al consenso in chat.

Atto 1 — La campagna nasce dalla chat (5 min) *(il motore della demo)*

Obiettivo: mostrare il tool **crm_a_phone_campaign**: l'agente aggancia la scheda alla **Pearl outbound già provisionata** (per nome, zero passi manuali su NLPearl), chiede conferma e parte la chiamata vera dell'Atto 2.

Azione: in chat:

"Crea una campagna outbound per il Galaxy 27, con una breve comparazione col modello precedente (S26), usando il numero outbound 686fd112a91849a9e59a5353, e riusa la pearl outbound esistente 'Campagna Galaxy S27'. Chiama chi preferisce il telefono."

Script (beat):

(*upsert con pearlName*) "L'agente crea la **scheda campagna**: prodotto, confronto col modello precedente, config telefonica (9–18, lun–ven, fuso Roma, 3 tentativi) — e la **aggancia alla Pearl outbound che abbiamo già**, per nome. Nessuna configurazione manuale su NLPearl, nessuna Pearl duplicata."

(*send* → *anteprima + conferma*) "Ora definisce *chi chiamare*: solo chi ha il consenso telefonico — qui **Lorenzo**. E **chiede a me la conferma** prima di mandare i lead." (*Confermare* → *leadsCreated: 1.*) "La campagna è pronta, la Pearl è ancora in pausa."

(*resume* → *conferma*) "Ora la attivo: la Pearl esistente comincia a chiamare. Confermo." (*Confermare.*)

👁 **Pubblico vede:** tool-call *upsert (pearlName)→send→resume*; scheda agganciata alla Pearl esistente; gate di conferma; *leadsCreated: 1.*

Regole sul palco: *send/resume* solo dietro conferma esplicita (*confirm: true*). Il numero outbound va sostituito con quello **verificato** per l'account; il nome della Pearl esistente con quello **già provisionato** (vedi Note NLPearl nel runbook).

Atto 2 — La prima chiamata vera: memoria + campagna in una chiamata (5 min)

🖥 **Azione:** confermare *resume* → il Pearl NLPearl chiama **davvero** il telefono del presentatore (numero di Lorenzo). Vivavoce; seguire il copione.

📞 **SCRIPT CHIAMATA OUTBOUND (Pearl campagna → Lorenzo):**

[AI] "Buongiorno Lorenzo, una chiamata per conto di Crm-A Console. Vorremmo presentarle una nuova offerta." [Lorenzo] "Buongiorno. Mi interessa — vi ho comprato il Galaxy S26, che offerta c'è per me?" [AI] (*dal Voice Brief*) "Il nuovo Galaxy S27 arriva il 18 ottobre: chip più veloce, batteria maggiore, fotocamera molto migliorata in bassa luce, sette anni di aggiornamenti. E per i possessori del Galaxy S26 c'è la promo di lancio: **fino a 150 euro di bonus permuta**." [Lorenzo] "Mi interessa. Potete ricontattarmi? E datemi pure la promo." [AI] (*raccoglie il consenso esplicitamente*) "Certamente. Confermo il suo consenso a ricevere comunicazioni di marketing e ad essere ricontattato per il lancio."

Preferisce il telefono?" [Lorenzo] "Sì, il telefono va bene." [AI] "Perfetto. La ricontatteremo al lancio. Grazie Lorenzo, buona giornata."

(L'AI improvvisa dal Voice Brief: battute attese, non copione cablato.)

Dopo la chiamata:

1. Dashboard NLPearl: chiamata completata, trascrizione + summary.
2. Console → profilo Lorenzo: **timeline** aggiornata con la nuova interazione **Call** (il webhook ha riconosciuto il numero e l'ha agganciato alla persona giusta).
3. Scheda campagna: stato lead → **Success**.
4. In chat: "Consolida i dati della chiamata: consenso marketing sì, canale preferito telefono." *(l'agente aggiorna il profilo; il consenso non si auto-scrive.)*

 **Script:** "Il telefono squilla davvero. E qui la Console riconosce il numero, lo aggancia a Lorenzo — non è un estraneo, è un cliente nato dall'acquisto — e registra l'esito in timeline. Poi io chiedo all'agente di consolidare il consenso: marketing sì, canale telefono. Un contatto pieno, con la sua storia."

 **Pubblico vede:** telefono che squilla + voce AI; call record NLPearl; interazione **Call**; campaign_send **Success**; profilo aggiornato.

Atto 3 — Il CRM diventa memoria (Copilot) (2 min)

 **Azione:** in chat:

"Stiamo lanciando il nuovo Samsung Galaxy. Qual è il pubblico migliore a cui proporlo?"

 **Script:** "Ora il CRM ragiona. Niente query scritte: l'agente usa la sua skill, legge segmento e consensi, e risponde. Ecco Lorenzo in lista — cliente S26, consenso sì, canale telefono. Una base dati *viva*, non un foglio Excel."

 **Pubblico vede:** risposta con la lista del pubblico e il ragionamento su canali e consensi.

Atto 4 — Orchestrazione multicanale: gli accessori (4 min)

Obiettivo: un **secondo caso d'uso distinto** dall'Atto 1: dopo il lancio del telefono, il CRM propone gli **accessori** (cover, auricolari, caricatore) al pubblico — sempre instradato per canale preferito. Stessa meccanica "brief → invio", ma su un **prodotto diverso** dal telefono.

 **Azione:** in chat:

"Prepara il brief degli accessori Galaxy per i clienti del lancio: cover S27, Galaxy Buds e caricatore rapido, con i prezzi e il link d'acquisto, e invialo al segmento 'Lancio Samsung Galaxy' per canale preferito."

 **Script (beat):**

(L'agente scrive `brief-accessori-galaxy.md` e lo apre.) "Il prodotto 'viaggia' come un brief di marketing — stavolta però **accessori**, non il telefono: cover, auricolari, caricatore. La stessa conoscenza che ha parlato al telefono nell'Atto 2, applicata a un'offerta nuova. Il CRM non è un foglio S27: è una base viva che sa proporre il prodotto giusto al momento giusto."

(L'agente lancia l'invio multicanale in **preview** e mostra la matrice di routing.) "Instradato **per canale preferito**: chi ha scelto Telegram la riceve su Telegram, chi email su email. Ecco chi finisce dove: `telegram: [Lorenzo, Marco]`, `email: [Giulia]`. Tutto in una schermata — nessun cambio di app."

 **Pubblico vede:** `brief-accessori-galaxy.md`; JSON { `ok`, `preview`, `telegram:[...]`, `email:[...]` } — la matrice di routing per canale, senza cambiare schermata.

Nota demo: si usa la **preview** (`preview: true`) — mostra il routing senza consegnare davvero. L'invio reale resta possibile (Telegram via runtime OpenClaw, email via SES) ed è da collaudare prima se si vuole mostrarlo dal vivo; se un canale fallisce in prova, narrarlo: "i canali live della demo sono il telefono — l'avete appena visto".

Beat opzionale — il loop Telegram in diretta (se il bot è in polling):

Dopo la preview, dimostrare che il canale Telegram è **bidirezionale** e che la Console riconosce il cliente:

1. *(Da un altro dispositivo/helper) il cliente scrive al bot: "Grazie, l'offerta mi interessa. Vale la pena?"*
2. Il **bridge inbound** (`message_received` → `POST /api/webhooks/phone` con `action: message`) risponde sul canale con il **contesto CRM**: "Cliente esistente: Lorenzo... ultimo acquisto: Samsung Galaxy S26... corriere GLS...". Lo stesso messaggio auto-mappa il `Telegram User ID` sul profilo.
3. *(Opzionale) In chat: "Manda un messaggio Telegram a Lorenzo" → tool `crm_a_telegram_person` → consegna su `telegram:<id>` (senza numero).*

Script:

"E il cliente può rispondere anche su Telegram. Il messaggio entra nella Console: riconosce chi è, gli risponde col contesto dell'ordine e in quel momento memorizza il suo Telegram User ID — così ora so come raggiungerlo anche senza il numero. Il canale che la preview vi ha appena mostrato, adesso vivo."

👁️ **Pubblico vede:** lo scambio sul bot (domanda → risposta con contesto CRM); il campo `Telegram User ID` compilato nel profilo; l'invio singolo dal comando chat.

Nota demo: richiede il bot Telegram in **polling** (`openclaw channels add telegram`). Se il bot non è disponibile, narrarlo con la preview: "i canali live della demo sono il telefono — l'avete appena visto".

Atto 5 — La campagna inbound nasce dalla chat (3 min) *(il servizio che risponde)*

Obiettivo: mostrare il secondo strumento telefonico: `crm_a_inbound_care`.

L'agente **attiva la Pearl inbound già provisionata** (per nome, zero passi manuali su NLPearl) — lo stesso servizio che risponderà a Lorenzo nell'Atto 6. Di qui parte il callback dell'Atto 6.

 **Azione:** in chat:

"Per il customer care inbound, riusa la pearl esistente 'Customer Care' sul numero inbound `686fd112a91849a9e59a5353`: saluta per nome chi già conosciamo, usa la memoria dell'ordine in consegna e proponi il brief del Galaxy 27. Attivala."

Script (beat):

(*activate con pearlName* → *conferma*) "Stesso principio dell'outbound, ma al contrario. L'agente aggancia la **Pearl inbound** che abbiamo già — la riconosce dal nome, nessuna creazione, nessuna configurazione manuale. Prima di salutare, una chiamata al CRM — il **PreCallAPI** — cerca il numero tra i contatti: se lo conosce, saluta per nome e legge lo stato dell'ordine; se non lo conosce, saluta genericamente."

(*Confermare.*) "Come per l'outbound, **l'agente mi chiede conferma** prima di attivare la linea. Confermo — da qui in poi quel numero risponde davvero." (→ *active: true.*)

 **Pubblico vede:** tool-call *activate (pearlName)*; Pearl inbound esistente che risponde; gate di conferma su *activate*.

Regole sul palco: *activate* solo dietro conferma esplicita (*confirm: true*). Il numero inbound e la Pearl esistente vanno collaudati **prima** di salire (mai la prima attivazione sul palco — vedi checklist).

Atto 6 — Il cliente richiama: la memoria risponde (5 min)

Obiettivo: il momento "wow": Lorenzo richiama e la Console lo riconosce — nome, ordine in consegna, e la nuova offerta.

 **Azione:** il presentatore, dallo stesso telefono, **chiama il numero inbound** (la Pearl inbound appena creata e attivata nell'Atto 5). Vivavoce; narrare prima e dopo; seguire il copione.

SCRIPT CHIAMATA INBOUND (Lorenzo → Pearl inbound):

[AI] "Buongiorno Lorenzo, come posso aiutarla?" [Lorenzo] "Buongiorno, è Crm-A? Volevo sapere del mio Galaxy S26 — dove è la consegna?" [AI] (*dal contesto*

restituito dal PreCallAPI) "Buongiorno Lorenzo. Il suo Galaxy S26 è in consegna con **GLS**: è in carico oggi, consegna prevista **domani entro le 18**. Posso aiutarla con altro?" **[Lorenzo]** "Perfetto. E sul nuovo Galaxy S27 che mi avete chiamato: vale la pena passare dal S26?" **[AI]** *(dal brief)* "Rispetto al suo S26: chip più veloce, batteria maggiore, fotocamera molto più performante in bassa luce, sette anni di aggiornamenti. Con la promo lancio per i possessori di S26 — fino a 150 euro di bonus permuta — il passaggio è vantaggioso." **[Lorenzo]** "Va bene, mi prenoto la promo. Procedete." **[AI]** "Perfetto Lorenzo, ho registrato il suo interesse per il Galaxy S27 con la promo di lancio. La ricontatteremo. Grazie e buona giornata!"

Dopo la chiamata — in chat:

"Lorenzo ha confermato la promo per il Galaxy 27. Registra l'ordine."

(L'agente registra l'ordine/pre-ordine del S27 con la promo.) Poi, per chiudere il cerchio:

"Cosa sappiamo di Lorenzo?"

(L'agente riassume: due ordini — S26 in consegna GLS + S27 pre-ordinato — consenso, canale, timeline.)

Script:

"E adesso il tocco finale. Lorenzo richiama — e ascoltate come il CRM lo riconosce: sa chi è, cosa ha comprato e dove è la consegna: 'in carico oggi, domani entro le 18'. Quella è la Pearl inbound che ho appena creato dall'Atto 5 — il PreCallAPI ha cercato il numero, l'ha trovato e ha personalizzato il saluto. Il brief del Galaxy 27 è lo stesso che ha parlato al telefono nell'Atto 2 — e ha chiuso il pre-ordine con la promo. Il cliente nato dieci minuti fa da un acquisto è ora una storia: due ordini, un consenso, due campagne, tre interazioni. **Niente più 'chi parla?'. È questo il CRM che ricorda.**"

 **Pubblico vede:** chiamata in vivavoce (saluto per nome + stato ordine); timeline che si aggiorna; comando di registrazione ordine; riassunto "cosa sappiamo di Lorenzo".

Atto 6bis — Variante senza telefono (simulata, per ambienti senza chiamata)

Usare quando non è possibile (o si vuole evitare) la chiamata vera. La Console non distingue un webhook simulato da una chiamata reale: il flusso è identico (PreCallAPI → riconoscimento → interazione in timeline), solo innescato da terminale invece che dal telefono.

 **Azione:** due comandi (verifica tecnica, in preparazione o durante la demo):

```
# 1) Il "PreCallAPI": la Console riconosce il numero di Lorenzo e
restituisce il contesto
#     che la Pearl inbound "pronuncerebbe".
curl -s "http://localhost:3100/api/nlpearl/precall?
token=$CRM_A_PHONE_WEBHOOK_SECRET&phone="+393312345678"
# → { data: { firstName: "Lorenzo", context: "Cliente esistente: Lorenzo...
ultimo acquisto:
#     Samsung Galaxy S26... corriere GLS... consegna prevista domani entro le
18..." } }
```

```
# 2) Webhook di fine chiamata: registra l'interazione Call in timeline
(stesso payload V2 che
#     NLPearl invierebbe a fine chiamata reale).
curl -sX POST "http://localhost:3100/api/nlpearl/webhook/call?
token=$CRM_A_PHONE_WEBHOOK_SECRET" \
-H 'content-type: application/json' -d '{
  "id":"demo-act6","pearlId":"<pearl-inbound-id>","startTime":"2026-10-
18T10:00:00Z",
  "conversationStatus":"Success","status":"Completed",

"from":"+393312345678","to":"+39654547159","name":"Lorenzo","duration":72,
  "summary":"Cliente interessato al Galaxy S27; confermata la promo di
lancio.",
  "leadId":null
}'
# → { ok, interactionId, personId } → la timeline di Lorenzo si aggiorna
con la Call.
```

 **Script:** identico all'Atto 6, con un'aggiunta: "la chiamata qui è simulata dal webhook, ma il flusso è lo stesso che la Pearl inbound esegue da sola: la Console cerca il numero, riconosce Lorenzo, e registra l'esito in timeline."

 **Pubblico vede:** contesto del PreCallAPI (riconoscimento + stato ordine) e la nuova interazione Call nella timeline di Lorenzo.

Chiusura (2 min) — Call to action

 **Script:**

"Ricapitolando. Il CRM partiva **vuoto**: il primo record è nato da un touchpoint — un acquisto sull'e-commerce che, via webhook, ha creato l'anagrafica, l'evento e l'ordine. L'operatore ha poi abilitato il consenso e in chat ha riusato **due servizi telefonici già pronti** — mai in automatico: la campagna outbound e l'agente inbound di customer care, agganciati per nome e attivati con conferma umana, senza toccare la dashboard. Il telefono ha squillato davvero: una voce vera, un consenso registrato. La stessa meccanica è diventata un **brief multicanale per gli accessori**, instradato per canale — un secondo caso d'uso, non una ripetizione. E quando il cliente ha richiamato, il CRM lo ha riconosciuto: nome, ordine, consegna, offerta — e ha chiuso il pre-ordine. Crm-A Console: **il CRM che ascolta, ricorda e agisce**. Domande?"

 **Azione:** aprire le domande. Tenere pronti `DEMO-RUNBOOK.md`, `SHOPIFY-SETUP.md` e `NLPEARL-SERVICES-PROMPT.md` per chi chiede come è costruito.

Raggiungibilità — Tailscale funnel (scelta: Tailscale)

La Console deve essere raggiungibile da NLPearl (callback) e da Shopify (webhook ordine). **Scelta: Tailscale funnel**. In produzione è attivo il funnel **sull'host** (nodo `top-mgm-00`, tailnet `taileb6b`) che pubblica la porta 3100 della Console:

```
# Sul host (già fatto — verificare con:)
tailscale funnel status
# → https://top-mgm-00-2.taileb6b.ts.net/ → proxy http://127.0.0.1:3100
```

Origin pubblica attuale: `https://top-mgm-00-2.taileb6b.ts.net` (già impostata come `CRM_A_CONSOLE_PUBLIC_URL` in `.env`).

Alternativa nel container (non usata ora): l'entrypoint Docker si unisce al tailnet e pubblica :3100 con `TAILSCALE_AUTHKEY+TAILSCALE_HOSTNAME` in `.env`; il log stampa `Tailscale funnel: https://crm-a-console.<tailnet>.ts.net`. Serve una auth key riutilizzabile e `funnel` abilitato nelle ACL per il nodo.

Checklist pre-demo (una scorsa prima di salire)

- ☐ `pnpm build:crm-a-plugins + docker compose up -d --build --force-recreate`
- ☐ `.env`: `CRM_A_PHONE_WEBHOOK_SECRET`, `NLPEARL_ACCOUNT_ID`, `NLPEARL_SECRET_KEY`, `TAILSCALE_AUTHKEY` (+ `TAILSCALE_HOSTNAME`), `SHOPIFY_API_SECRET`, `SHOPIFY_STORE_DOMAIN`
- ☐ **Funnel attivo**: `https://top-mgm-00-2.taileb6b.ts.net/api/integrations` → 200
- ☐ **Shopify**: dev store + prodotto SAM-S26 + app custom con webhook `orders/create` e `order/fulfilled` → URL Console (vedi `SHOPIFY-SETUP.md`)
- ☐ **Seed + reset**: `bash scripts/demo-seed.sh` → crea catalogo (SAM-S27/S26/S25) + 4 contatti + ordine Lorenzo + segmento "Lancio Samsung Galaxy", poi **rimuove Lorenzo** (persona+ordine). Verifica finale: 3 prodotti, 3 contatti (senza Lorenzo), 1 segmento.
- ☐ Numero inbound **verificato** (`686fd112a91849a9e59a5353`); numero outbound **verificato**; chiamata outbound **collaudata** (mai la prima volta sul palco)
- ☐ **Collaudo inbound prima del palco**: `activate` con `pearlName` della Pearl esistente + una chiamata inbound reale di prova — poi `pause` e lasciare pulita la dashboard
- ☐ **Nomi Pearl esistenti** annotati (outbound "Campagna Galaxy S27" e inbound "Customer Care") — i tool li risolvono per nome (`pearlName`)
- ☐ Collaudo webhook Shopify con `scripts/shopify-demo-simulate.sh` (+ `--fulfilled`)
- ☐ Canali Telegram/email (Atto 4) collaudati — o script pronto a narrarne i `failed: [...]`
- ☐ **Bot Telegram in polling** (`openclaw channels add telegram` — il token da solo non riceve) + **bridge inbound collaudato**: messaggio al bot → risposta col contesto CRM → `Telegram User ID` auto-mappato nel profilo
- ☐ Pearl residue di collaudo rimosse dalla dashboard NLPearl
- ☐ Telefono carico, in vivavoce, numeri (inbound/outbound) a portata di mano; hard refresh browser